🤖 PROJECT STARTER PACK

Project 1: Smart Temperature Monitor with Auto Alert

Paste this entire document into a new Claude chat as your first message.
It gives Claude full context of who you are and what you need.

A. WHO I AM — Context for Claude

📌 Copy everything in this document and paste it as your FIRST message in a new Claude conversation. Claude will instantly know your full background and guide you perfectly.

My Background:

- Name: Student at Malla Reddy Vishwavidyapeetham (NIAT collab), Hyderabad
- Degree: B.Tech CS-IT — Year 1 (2025–2026), transitioning from EEE Diploma (2022–2025)
- EEE Diploma: Embedded systems, digital electronics, AC/DC machines, basic circuits — I understand hardware
- Location: Andhra Pradesh, India
- Goal: AIoT Engineer → Remote internship 2026 → Japan job 2029 → Vizag startup 2033

My Current Skills (honest self-assessment):

Skill	Level	Details
DSA in C++	Beginner	Learning Striver A2Z sheet, done arrays basics
Python	Beginner-Intermediate	Functions, loops, file I/O — not yet NumPy/PyTorch
Linux CLI	Beginner	Basic navigation, not yet scripting
MERN Stack	Basic awareness	Know the concepts, not built anything yet
MySQL / MongoDB	Basic	Know queries, not yet used in a project
Japanese	N5 level	Know Hiragana, Katakana, 300 vocab words
Hardware	Strong (EEE)	GPIO, I2C, SPI, sensors, circuits — this is my advantage

My Study Schedule (daily):

- 4:00–4:30 AM: Wake up and fresh up
- 5:00–6:00 AM: Workout
- 6:30 AM–4:00 PM: College (Malla Reddy)
- 4:00–5:15 PM: Lunch and nap
- 5:30–9:30 PM: Study session — split between DSA, Python/AI, Web, Japanese

- 9:30–10:00 PM: Review college notes and GitHub commit
- Sundays: AI automation learning (Cursor, Claude API, n8n, LangChain)

Tools I Have Available (all free):

- Claude.ai — free tier (this chat)
- GitHub Copilot — free via GitHub Student Developer Pack (education.github.com/students)
- Cursor IDE — free tier (2000 completions/month)
- Google Gemini — free (gemini.google.com)
- Raspberry Pi 4 — available through NIAT lab
- VSCode on laptop
- Ubuntu/Linux — either WSL2 on Windows or dual boot

How I want Claude to help me:

- Explain concepts simply — I am a beginner, use analogies before technical details
- Give me step-by-step instructions — number every step
- Tell me EXACTLY what command to type — no vague instructions
- When I paste an error — tell me exactly what it means and exactly how to fix it
- When I am stuck — give me a hint first, not the full solution
- Keep explanations short — I am on mobile, short paragraphs please
- If I ask something risky (like rm -rf) — warn me first

B. PROJECT 1 OVERVIEW

Smart Temperature Monitor with Auto Email Alert

⏱️ Estimated Time: 3–4 hours 🎯 Difficulty: Beginner 📍 Platform: Raspberry Pi 4

This is my very first IoT project. A DHT22 temperature and humidity sensor is connected to Raspberry Pi GPIO. A Python script reads the sensor every 10 seconds. If temperature goes above a set limit, it automatically sends me an email alert. This teaches me the complete sensor → Python → alert pipeline.

B.1 What I Will Learn

- How to connect a physical sensor to Raspberry Pi GPIO pins
- How to read sensor data using a Python library
- How to send emails from Python using smtplib
- How to use environment variables (.env file) to store passwords safely
- How to add error handling for hardware failures
- How to use logging instead of print() — professional habit
- How to run a Python script permanently in the background

B.2 Hardware Required

Component	Quantity	Where to Get	Approximate Cost
Raspberry Pi 4 (2GB or 4GB)	1	NIAT lab / Electronics store	Available in lab
DHT22 temperature+humidity sensor	1	Robu.in / Amazon India	₹150–200
Female-to-female jumper wires	3	Robu.in / Electronics store	₹30–50
Breadboard (optional)	1	Any electronics shop	₹50–80
10kΩ resistor (optional but better)	1	Any electronics shop	₹2

> 📌 If DHT22 is not available, DHT11 also works. DHT11 is less accurate but costs only ₹50. The code change is one word: adafruit_dht.DHT11 instead of adafruit_dht.DHT22.

B.3 Software Required

Library/Tool	Purpose	Install Command
Python 3.11+	Programming language	Already on Raspberry Pi OS
adafruit-circuitpython-dht	Read DHT22 sensor	pip3 install adafruit-circuitpython-dht
adafruit-blinka	Hardware abstraction layer	pip3 install adafruit-blinka
python-dotenv	Read .env file safely	pip3 install python-dotenv
smtplib	Send emails	Built into Python — no install needed
logging	Professional log messages	Built into Python — no install needed

B.4 Wiring Diagram — Exactly How to Connect

> ⚠️ Before wiring anything: power off the Raspberry Pi completely. Never connect wires while Pi is on.

DHT22 has 3 or 4 pins. Look at the flat face of the sensor (the side with the holes):

```
DHT22 Pin Layout (looking at flat/front face):

  [PIN 1]  [PIN 2]  [PIN 3]  [PIN 4]
   VCC      DATA     NC       GND
  (power)  (signal) (empty)  (ground)

If your DHT22 is a module (on a small PCB board):
  It has only 3 pins: VCC, DATA, GND — ignore NC
```

DHT22 Pin	Connect To	Raspberry Pi Pin Number	Wire Colour (suggested)
VCC (Pin 1)	3.3V power	Physical Pin 1	Red
DATA (Pin 2)	GPIO4	Physical Pin 7	Yellow or Green
GND (Pin 4)	Ground	Physical Pin 6	Black
10kΩ resistor	Between VCC and DATA	Not a Pi pin — on breadboard	N/A

📌 The 10kΩ resistor is called a "pull-up resistor". It helps the DATA signal stay stable. Without it the sensor still works but may give occasional errors. Place one leg of the resistor in the same breadboard row as DHT22 VCC, and the other leg in the same row as DHT22 DATA.

```
Raspberry Pi 4 Pin Reference (physical pin numbers):

 [Pin 1 = 3.3V]  [Pin 2 = 5V    ]
 [Pin 3 = GPIO2] [Pin 4 = 5V    ]
 [Pin 5 = GPIO3] [Pin 6 = GND   ] ← connect DHT GND here
 [Pin 7 = GPIO4] [Pin 8 = GPIO14] ← connect DHT DATA here

 Run this command to see Pi pin layout:
 pinout
```

C. COMPLETE PROJECT CODE

C.1 Folder Structure

```
project1/
├── monitor.py        ← main script (runs forever)
├── .env              ← your email credentials (NEVER share this file)
├── .gitignore        ← tells Git to ignore .env
└── README.md         ← project documentation
```

C.2 File 1 — .env (your credentials)

⚠️ This file contains your email password. NEVER upload it to GitHub. The .gitignore file below protects you.

```
# .env — create this file in your project1/ folder
# Replace the values with your actual email details

EMAIL_FROM=youremail@gmail.com
EMAIL_PASSWORD=your_gmail_app_password_here
EMAIL_TO=youremail@gmail.com
ALERT_TEMP=35.0
CHECK_INTERVAL=10
```

📌 Gmail requires an "App Password" — not your normal Gmail password. Go to: myaccount.google.com → Security → 2-Step Verification (enable first) → App Passwords → Generate one for "Mail". Use that 16-character code as EMAIL_PASSWORD.

C.3 File 2 — .gitignore

```
# .gitignore — create this file in project1/ folder
# These files will NOT be uploaded to GitHub

.env
__pycache__/
*.pyc
*.log
```

C.4 File 3 — monitor.py (main script)

This is the complete, production-quality main script. Every line is commented to explain what it does and WHY.

```
#!/usr/bin/env python3
```

```
"""
monitor.py — Smart Temperature Monitor with Email Alert
Project 1 of AIoT Engineer study plan

What this does:
  - Reads DHT22 temperature and humidity every N seconds
  - If temperature exceeds ALERT_TEMP, sends an email
  - Uses a 30-minute cooldown to prevent email spam
  - Logs all readings to console and a log file
  - Handles sensor errors gracefully (DHT sensors fail occasionally)
"""

import time
import smtplib
import logging
import os
from email.mime.text import MIMEText
from email.mime.multipart import MIMEMultipart
from datetime import datetime
from dotenv import load_dotenv

# adafruit_dht needs adafruit_blinka installed first
import adafruit_dht
import board

# ── STEP 1: Load credentials from .env file ──────────────────
# load_dotenv() reads the .env file and makes values available
# via os.getenv(). This keeps passwords out of your code.
load_dotenv()

EMAIL_FROM      = os.getenv("EMAIL_FROM")
EMAIL_PASSWORD  = os.getenv("EMAIL_PASSWORD")
EMAIL_TO        = os.getenv("EMAIL_TO")
ALERT_TEMP      = float(os.getenv("ALERT_TEMP", "35.0"))
CHECK_INTERVAL  = int(os.getenv("CHECK_INTERVAL", "10"))
ALERT_COOLDOWN  = 1800  # 30 minutes between alerts (in seconds)

# ── STEP 2: Set up logging ────────────────────────────────────
# logging is better than print() because:
# - it adds timestamps automatically
# - you can save logs to a file
# - you can set different levels (DEBUG, INFO, WARNING, ERROR)
logging.basicConfig(
    level=logging.INFO,
    format="%(asctime)s [%(levelname)s] %(message)s",
    datefmt="%Y-%m-%d %H:%M:%S",
```

```
    handlers=[
        logging.StreamHandler(),              # print to terminal
        logging.FileHandler("monitor.log"),   # also save to file
    ]
)
logger = logging.getLogger(__name__)

# ── STEP 3: Connect to DHT22 sensor ──────────────────────────
# board.D4 means GPIO pin 4 on the Raspberry Pi
# If you used a different GPIO pin, change D4 to that pin number
# Examples: board.D17  board.D27  board.D22
try:
    sensor = adafruit_dht.DHT22(board.D4)
    logger.info("DHT22 sensor initialised on GPIO4")
except Exception as e:
    logger.error(f"Failed to initialise sensor: {e}")
    logger.error("Check wiring: VCC→Pin1, DATA→Pin7, GND→Pin6")
    raise SystemExit(1)  # stop the program if sensor not found

# ── STEP 4: Email sending function ───────────────────────────
def send_alert_email(temperature: float, humidity: float) -> bool:
    """
    Send an email alert when temperature is too high.
    Returns True if sent successfully, False if failed.
    """
    try:
        # Build the email
        msg = MIMEMultipart()
        msg["From"]    = EMAIL_FROM
        msg["To"]      = EMAIL_TO
        msg["Subject"] = f"🌡️ ALERT: Temperature {temperature:.1f}°C"

        body = f"""
⚠️ HIGH TEMPERATURE ALERT

Temperature : {temperature:.1f}°C  (Limit: {ALERT_TEMP}°C)
Humidity    : {humidity:.1f}%
Time        : {datetime.now().strftime("%Y-%m-%d %H:%M:%S")}
Device      : Raspberry Pi — DHT22 Sensor

Please check your system.
        """
        msg.attach(MIMEText(body, "plain"))

        # Send via Gmail SMTP
        # SMTP_SSL means the connection is encrypted (secure)
```

```
        with smtplib.SMTP_SSL("smtp.gmail.com", 465) as server:
            server.login(EMAIL_FROM, EMAIL_PASSWORD)
            server.sendmail(EMAIL_FROM, EMAIL_TO, msg.as_string())

        logger.info(f"Alert email sent to {EMAIL_TO}")
        return True

    except smtplib.SMTPAuthenticationError:
        logger.error("Email authentication failed!")
        logger.error("Check: EMAIL_FROM and EMAIL_PASSWORD in .env")
        logger.error("Make sure you used Gmail App Password, not normal password")
        return False

    except smtplib.SMTPException as e:
        logger.error(f"SMTP error sending email: {e}")
        return False

    except Exception as e:
        logger.error(f"Unexpected error sending email: {e}")
        return False

# ── STEP 5: Validation ──────────────────────────────────────
def validate_config() -> None:
    """Check that all required settings are present before starting."""
    missing = []
    if not EMAIL_FROM:     missing.append("EMAIL_FROM")
    if not EMAIL_PASSWORD: missing.append("EMAIL_PASSWORD")
    if not EMAIL_TO:       missing.append("EMAIL_TO")
    if missing:
        logger.error(f"Missing in .env file: {missing}")
        raise SystemExit(1)
    logger.info(f"Config OK — alert if temp > {ALERT_TEMP}°C, check every
{CHECK_INTERVAL}s")

# ── STEP 6: Main monitoring loop ────────────────────────────
def main() -> None:
    """Main loop — runs forever until Ctrl+C."""
    validate_config()
    logger.info("Smart Temperature Monitor started. Press Ctrl+C to stop.")
    logger.info(f"Sending alerts to: {EMAIL_TO}")

    last_alert_time = 0  # timestamp of last alert (0 = never sent)
    reading_count   = 0  # count total readings for stats
    error_count     = 0  # consecutive errors (stop if too many)

    while True:
```

```
        try:
            # Read sensor values
            temperature = sensor.temperature  # degrees Celsius
            humidity    = sensor.humidity     # percent

            # Validate reading (sensor sometimes returns None)
            if temperature is None or humidity is None:
                raise RuntimeError("Sensor returned None — try again")

            reading_count += 1
            error_count    = 0  # reset on success

            # Log every reading
            logger.info(
                f"Reading #{reading_count:04d} | "
                f"Temp={temperature:.1f}°C | "
                f"Humidity={humidity:.1f}%"
            )

            # Check if alert needed
            current_time   = time.time()
            time_since_last = current_time - last_alert_time

            if temperature > ALERT_TEMP:
                if time_since_last >= ALERT_COOLDOWN:
                    logger.warning(
                        f"HIGH TEMP: {temperature:.1f}°C > limit {ALERT_TEMP}°C"
                    )
                    if send_alert_email(temperature, humidity):
                        last_alert_time = current_time
                else:
                    mins_left = int((ALERT_COOLDOWN - time_since_last) / 60)
                    logger.info(
                        f"High temp but cooldown active — next alert in {mins_left}
min"
                    )

        except RuntimeError as e:
            # DHT sensors fail 5-10% of the time — this is normal
            error_count += 1
            logger.warning(f"Sensor read failed ({error_count}): {e}")
            if error_count >= 10:
                logger.error("10 consecutive failures — check wiring!")
                error_count = 0  # reset and keep trying

        except KeyboardInterrupt:
```

```
            # User pressed Ctrl+C to stop
            logger.info(f"Stopped. Total readings: {reading_count}")
            break

        time.sleep(CHECK_INTERVAL)


if __name__ == "__main__":
    main()
```

C.5 File 4 — README.md

````
# Project 1: Smart Temperature Monitor

Reads DHT22 sensor every 10 seconds. Sends email alert if
temperature exceeds limit. Runs on Raspberry Pi 4.

## Hardware
- Raspberry Pi 4
- DHT22 sensor
- 3 jumper wires

## Wiring
| DHT22 Pin | Raspberry Pi Pin |
|-----------|------------------|
| VCC       | Pin 1 (3.3V)     |
| DATA      | Pin 7 (GPIO4)    |
| GND       | Pin 6 (GND)      |

## Setup
```bash
pip3 install adafruit-circuitpython-dht adafruit-blinka python-dotenv
cp .env.example .env   # fill in your email credentials
python3 monitor.py
```

## What I Learned
- DHT22 sensor reading via adafruit library
- Email sending with smtplib
- Secure credential storage with .env
- Error handling for hardware failures
- Professional logging
````

D. STEP-BY-STEP SETUP INSTRUCTIONS

D.1 Prepare Raspberry Pi

Step	Command / Action	Expected Result
1	Connect Pi to monitor/TV via HDMI, plug in keyboard	Pi boots to desktop or terminal
2	Open terminal (Ctrl+Alt+T or click terminal icon)	Black terminal window opens
3	Update Pi: sudo apt update && sudo apt upgrade -y	Downloads updates (takes 2–5 min)
4	Check Python: python3 --version	Should show Python 3.9 or higher
5	Enable I2C and SPI: sudo raspi-config → Interfaces	Enable I2C, SPI, and Camera

D.2 Install Required Libraries

```
# Run these commands in terminal on Raspberry Pi
# Each line is a separate command — press Enter after each

# Update pip first
pip3 install --upgrade pip

# Install sensor library
pip3 install adafruit-circuitpython-dht

# Install hardware abstraction layer (needed by adafruit)
pip3 install adafruit-blinka

# Install .env file reader
pip3 install python-dotenv
```

📌 If you see "Permission denied" errors, add sudo before pip3. Example: sudo pip3 install adafruit-circuitpython-dht

D.3 Create Project Folder and Files

```
# Create project folder
mkdir ~/project1
cd ~/project1

# Create each file
nano .env           # paste the .env content from Section C.2
```

```
nano .gitignore     # paste the .gitignore content from Section C.3
nano monitor.py     # paste the full code from Section C.4

# In nano editor:
#   Ctrl+X = exit
#   Y = yes save
#   Enter = confirm filename
```

D.4 Set Up Gmail App Password

- Step 1: Open browser on any device → go to myaccount.google.com
- Step 2: Click "Security" in left menu
- Step 3: Scroll to "How you sign in to Google" → click "2-Step Verification"
- Step 4: Enable 2-Step Verification if not already on
- Step 5: Go back to Security page → scroll down → find "App passwords"
- Step 6: Select app: Mail → Select device: Other → type "RaspberryPi" → Generate
- Step 7: Copy the 16-character code (like: abcd efgh ijkl mnop)
- Step 8: Paste this code (without spaces) as EMAIL_PASSWORD in your .env file

⚠️ Never use your actual Gmail password. Always use the App Password. If you share your .env file by accident, go to myaccount.google.com and revoke that App Password immediately.

D.5 Wire the Sensor (do this with Pi powered OFF)

- Step 1: Power off Pi completely — sudo shutdown now
- Step 2: Disconnect power cable from Pi
- Step 3: Connect DHT22 VCC → Pi Physical Pin 1 (red wire)
- Step 4: Connect DHT22 GND → Pi Physical Pin 6 (black wire)
- Step 5: Connect DHT22 DATA → Pi Physical Pin 7 / GPIO4 (yellow wire)
- Step 6: Double-check all connections are firm
- Step 7: Reconnect power cable — Pi boots

D.6 Test the Sensor First

```
# Before running the full script, test the sensor alone
cd ~/project1
python3 -c "
import adafruit_dht, board
sensor = adafruit_dht.DHT22(board.D4)
print(f'Temp: {sensor.temperature}°C, Humidity: {sensor.humidity}%')
"

# Expected output:
# Temp: 27.3°C, Humidity: 62.5%
```

```
# If you see an error like "RuntimeError: DHT sensor not found":
#   → Check wiring. Most common: DATA wire on wrong pin
#   → Try again — DHT sensors sometimes fail on first read
```

D.7 Run the Full Script

```
cd ~/project1
python3 monitor.py

# You should see output like:
# 2026-09-07 17:35:00 [INFO] DHT22 sensor initialised on GPIO4
# 2026-09-07 17:35:00 [INFO] Config OK — alert if temp > 35.0°C
# 2026-09-07 17:35:00 [INFO] Smart Temperature Monitor started.
# 2026-09-07 17:35:10 [INFO] Reading #0001 | Temp=27.3°C | Humidity=62.5%
# 2026-09-07 17:35:20 [INFO] Reading #0002 | Temp=27.4°C | Humidity=62.3%

# To test the email alert:
# Change ALERT_TEMP=35.0 in .env to ALERT_TEMP=20.0
# (current room temp is probably above 20°C)
# Restart the script — you should receive an email within 30 seconds
```

D.8 Make It Run Automatically on Boot

```
# Create a systemd service so monitor starts when Pi boots
sudo nano /etc/systemd/system/temperature-monitor.service

# Paste this content:
[Unit]
Description=Temperature Monitor
After=network.target

[Service]
Type=simple
User=pi
WorkingDirectory=/home/pi/project1
ExecStart=/usr/bin/python3 /home/pi/project1/monitor.py
Restart=always
RestartSec=10

[Install]
WantedBy=multi-user.target

# Save (Ctrl+X, Y, Enter), then enable:
sudo systemctl daemon-reload
```

```
sudo systemctl enable temperature-monitor
sudo systemctl start temperature-monitor

# Check status:
sudo systemctl status temperature-monitor

# View live logs:
sudo journalctl -u temperature-monitor -f
```

E. COMMON ERRORS AND EXACT FIXES

When something goes wrong, paste the FULL error message into Claude. Below are the most common errors and their fixes.

Error Message	What It Means	Exact Fix
ModuleNotFoundError: No module named adafruit_dht	Library not installed	Run: pip3 install adafruit-circuitpython-dht
RuntimeError: DHT sensor not found, check wiring	Sensor not connected properly	Check: VCC→Pin1, DATA→Pin7, GND→Pin6. Try again — DHT sometimes fails first read
RuntimeError: Checksum did not validate	Sensor read error (normal 5% of time)	Script handles this automatically. If happens >50% of time, check wiring
SMTPAuthenticationError	Wrong email or password	Make sure you used App Password, not normal Gmail password
PermissionError: [Errno 13]	Python not allowed to use GPIO	Run: sudo pip3 install adafruit-blinka — then reboot
FileNotFoundError: .env	.env file not in same folder	Make sure .env is in project1/ folder with monitor.py
AttributeError: temperature is None	Sensor returned empty reading	Script handles this. If frequent, add 10kΩ pull-up resistor
Connection refused (SMTP)	Network issue or firewall	Check Pi internet: ping google.com

F. PUSH TO GITHUB — YOUR FIRST REPO

Every project must be on GitHub. Recruiters check your GitHub profile. This is your proof of work.

F.1 One-Time Git Setup (do this once on Pi)

```
git config --global user.name "Your Name"
git config --global user.email "youremail@gmail.com"
```

F.2 Create GitHub Repo and Push

- Step 1: Go to github.com → New Repository
- Step 2: Name: smart-temperature-monitor
- Step 3: Description: "IoT temperature monitor with email alert — Raspberry Pi + DHT22 + Python"
- Step 4: Set to Public, do NOT add README (we already have one)
- Step 5: Click "Create repository"
- Step 6: Copy the repo URL (looks like: https://github.com/username/smart-temperature-monitor.git)

```
# On Raspberry Pi terminal:
cd ~/project1
git init
git add .
git status   # verify .env is NOT listed (gitignore should hide it)
git commit -m "feat: smart temperature monitor with email alert"
git branch -M main
git remote add origin https://github.com/YOUR_USERNAME/smart-temperature-monitor.git
git push -u origin main
```

⚠️ Run git status before git add . — confirm .env is NOT in the list. If it appears, your .gitignore is not working. Fix it before pushing. Once pushed to GitHub it is public forever.

G. EXACT CLAUDE PROMPTS — COPY AND PASTE THESE

These are the exact prompts to use in your new Claude conversation for this project. Copy these word for word.

G.1 Starting Prompt (paste this first in new chat)

```
I am starting Project 1 from my AIoT study plan. Here is my full background context: [paste this entire document here]. I am ready to start building. Guide me through the setup step by step. I am on a Raspberry Pi 4 with Raspberry Pi OS. Ask me one question at a time and wait for my answer before moving to the next step.
```

G.2 When You Hit an Error

```
I got this error while running monitor.py: [paste full error here]. I am on Raspberry Pi 4, Python 3.11, DHT22 connected to GPIO4. What exactly went wrong and what is the exact command to fix it?
```

G.3 When Sensor Gives Wrong Values

```
My DHT22 sensor is giving readings that seem wrong. Temperature shows 85°C and humidity shows 0%. The room is around 27°C. What are the possible causes and how do I debug this?
```

G.4 When Email Is Not Sending

```
The email alert is not being sent. I can see the WARNING log saying "HIGH TEMP" but no email arrives. My .env has EMAIL_FROM, EMAIL_PASSWORD (App Password), EMAIL_TO all filled in. The error says [paste error]. What should I check?
```

G.5 When You Want to Extend the Project

```
Project 1 is working. Temperature monitor is running and email alerts work. Now I want to add: [choose one] (a) save readings to a CSV file, OR (b) also alert if humidity goes above 80%, OR (c) show a graph of last 1 hour of readings. Which is easiest to add first and what is the code?
```

G.6 For Understanding a Concept

```
In monitor.py line 67, why do we use "with smtplib.SMTP_SSL" instead of just smtplib.SMTP? What is the difference? Explain simply using an analogy.
```

G.7 For Code Review

```
I finished Project 1 and it is working. Can you review my monitor.py code and
tell me: (1) what I did well, (2) what can be improved, (3) is there anything
that could break in production? Here is my code: [paste your code]
```

H. RECOMMENDED CLAUDE SETTINGS

These settings make Claude more useful for your specific situation. Set these once.

H.1 User Preferences (in Claude settings)

Go to claude.ai → click your name/profile → Settings → Profile. In the "What should Claude know about you?" box, paste this:

```
I am a B.Tech CS-IT student at Malla Reddy University, Hyderabad. I have an EEE diploma background so I understand hardware. I am learning AIoT engineering — building projects with Raspberry Pi, Python, TFLite, MQTT, and Claude API. My goal is to become an Edge AI / AIoT engineer and work in Japan by 2029. I am a beginner in Python and Linux but strong in hardware. I study in the evenings 5:30-9:30 PM. I am using free tools only (no paid subscriptions). I prefer: short paragraphs (I am on mobile), numbered steps for instructions, analogies before technical details, exact commands to type.
```

📌 This profile is saved permanently. Every new Claude conversation automatically knows your background. You do not need to re-explain yourself each time.

H.2 How to Start Every New Project Chat

At the start of every new project conversation, paste this short opener:

```
I am starting [Project Name] from my AIoT study plan. My background is in the profile. Today is [date]. I am on [laptop/Raspberry Pi]. Current skill level for this project: [beginner/familiar]. Let us start. What is the first thing I need to do?
```

H.3 Settings to Enable in Claude

Setting	Where to Find It	Enable / Disable	Why
Web Search	In chat — toggle button	Enable	Gets latest library versions and error solutions
Code Execution	In chat — toggle button	Enable if available	Claude can test small code snippets
Memory (if available)	Settings → Memory	Enable	Remembers your past projects across conversations
Long responses	User Preferences	Ask for detailed responses	For project guides you want complete answers

H.4 One Prompt Style Tip

Claude gives much better answers when you give context. Compare:

> ⚠️ Bad: "why is my code not working"
>
> 💡 Good: "My monitor.py gives RuntimeError: DHT sensor not found on line 45. I am on Raspberry Pi 4, GPIO pin 4, DHT22 sensor. I wired VCC→Pin1, DATA→Pin7, GND→Pin6. What is wrong?"

The more specific you are, the more useful the answer. Always include: what you are building, what you tried, what exact error you see, what you expected to happen.

I. PROJECT COMPLETION CHECKLIST

Before calling this project "done", make sure all items are checked:

Checklist Item	Status
DHT22 sensor wired correctly — VCC, DATA, GND	☐ Done
Libraries installed: adafruit-dht, adafruit-blinka, python-dotenv	☐ Done
Gmail App Password created and in .env file	☐ Done
.gitignore file created (protects .env from GitHub)	☐ Done
Sensor test passes: python3 -c "print(sensor.temperature)"	☐ Done
monitor.py runs without errors	☐ Done
Email alert received when temperature exceeds limit	☐ Done
monitor.log file created with readings	☐ Done
Systemd service created — auto-starts on boot	☐ Done
Code pushed to GitHub — repo is public	☐ Done
README.md written with wiring table and setup steps	☐ Done
LinkedIn post written about completing the project	☐ Done

You are ready to start. Paste this entire document into a new Claude chat.

Good luck with Project 1. 🚀